

OOP basics

COMPLETE LEARNING GUIDE

What You'll Learn

Java is fundamentally an object-oriented language, and its four pillars — encapsulation, inheritance, polymorphism, and abstraction — are asked about in nearly every technical interview, often with a request to explain each using a concrete example rather than a textbook definition. This guide walks through how classes and objects actually work in Java's memory model, the access modifiers that enforce encapsulation, how inheritance and the `super` keyword let one class extend another, the crucial difference between compile-time polymorphism (overloading) and runtime polymorphism (overriding), and when to reach for an abstract class versus an interface. It closes with the `equals/hashCode/toString` contract, a topic that trips up even experienced developers when they override one but forget the other, silently breaking hash-based collections like `HashMap` and `HashSet`.

1. Classes, Objects, and Constructors

A class is a blueprint describing state (fields) and behavior (methods); an object is a runtime instance of that blueprint, created with `new`, which allocates memory on the heap and invokes a constructor. A constructor shares its name with the class, has no return type (not even `void`), and its job is to put the new object into a valid initial state. If no constructor is written, Java supplies a no-argument default constructor automatically — but only if zero constructors exist; the moment you write any constructor yourself, the implicit default disappears, a frequent source of "constructor not found" compile errors when a no-arg constructor is expected elsewhere (like frameworks that instantiate via reflection).

JAVA

```
public class Employee {
    private String name;
    private double salary;

    public Employee(String name, double salary) {
        this.name = name;        // 'this' disambiguates field from parameter
        this.salary = salary;
    }
}
```

2. Encapsulation and Access Modifiers

Encapsulation means bundling data with the methods that operate on it and restricting direct access to that data from outside the class — typically by making fields private and exposing controlled access through public getter/setter methods, which lets the class validate input or change its internal representation later without breaking callers. Java has four access levels applied to classes, fields, and methods, and interviewers routinely ask you to rank them by visibility, since "package-private" (no modifier at all) is the one candidates most often forget exists.

- private — visible only within the same class.
- (no modifier, "package-private") — visible within the same package only.
- protected — visible within the same package, plus subclasses in other packages.
- public — visible everywhere.

3. Inheritance and super

A class extends another using the extends keyword, inheriting its public and protected members and gaining the ability to add new fields/methods or override existing ones. Java supports single inheritance of classes only (a class can extend exactly one superclass) precisely to avoid the "diamond problem" ambiguity that multiple class inheritance creates — multiple inheritance of behavior is instead achieved through interfaces. The super keyword refers to the immediate superclass: super(...) calls a specific superclass constructor and must be the first statement in a subclass constructor if used, while super.method() invokes the superclass's version of an overridden method from within the override.

JAVA

```
public class Manager extends Employee {
    private int teamSize;

    public Manager(String name, double salary, int teamSize) {
        super(name, salary);    // must be the first statement
        this.teamSize = teamSize;
    }

    @Override
    public String toString() {
        return super.toString() + ", team size: " + teamSize;
    }
}
```

4. Polymorphism: Overloading vs. Overriding

Method overloading is compile-time (static) polymorphism: multiple methods in the same class share a name but differ in parameter list (number, type, or order), and the compiler picks which one to call based on the argument types at the call site. Method overriding is runtime (dynamic) polymorphism: a subclass redefines a method it inherited with the identical signature, and which version actually runs is decided at runtime based on the object's actual class, not the reference's declared type — this is why a `Employee e = new`

Manager(...); e.toString() call invokes Manager's overridden version, not Employee's. The @Override annotation is not required by the compiler but should always be used, because it makes the compiler verify the signature genuinely matches a superclass method, catching accidental overloads (a typo'd parameter type) that would otherwise silently fail to override anything.

- Overloading: same method name, different parameter list, resolved at compile time.
- Overriding: same signature in a subclass, resolved at runtime based on the actual object type.
- @Override catches accidental overloads caused by a mismatched signature — always use it.
- Static methods, private methods, and fields are NOT polymorphic — they are resolved by the reference's declared type, not the object's actual type.

5. Abstraction: Abstract Classes vs. Interfaces

An abstract class can hold both fully implemented methods and abstract (unimplemented) ones, can declare fields with any access modifier, and is extended by exactly one subclass at a time — it models an "is-a" relationship where subclasses share substantial common implementation. An interface (in modern Java) can hold abstract methods, default methods with a body, static methods, and constants (implicitly public static final), and a class can implement any number of interfaces — it models a "can-do" capability contract rather than an inheritance hierarchy. The practical rule interviewers look for: use an abstract class when subclasses share meaningful state and implementation; use an interface when unrelated classes need to guarantee the same capability.

6. Static vs. Instance Members

A static field or method belongs to the class itself, shared by every instance and accessible without creating an object (ClassName.member); an instance field or method belongs to a specific object, and each object gets its own copy of instance fields. Static methods cannot access instance (non-static) members directly, because there's no implicit this — there might be zero, one, or a thousand instances in existence when a static method runs. A common beginner mistake is marking a method static purely to "call it without new," without considering whether it logically needs per-object state, leading to designs that can't be polymorphic since static methods aren't overridden, only hidden.

7. The equals(), hashCode(), and toString() Contract

By default, Object.equals() compares references (identical to ==) and Object.hashCode() derives from the object's memory address, which means two logically "equal" objects (two Point objects both at (1,2)) are not equal by default and won't behave correctly as HashMap keys or HashSet elements. The contract requires: if two objects are equal via .equals(), they MUST return the same hashCode() — violating this silently breaks hash-based collections, because an object can be inserted into a HashSet and then become "unfindable" by an equal object with a different hash. IDEs and modern Java (records, Lombok) can generate a correct equals/hashCode pair automatically, and when writing it by hand, always override both together, never just one.

- Override `equals()` and `hashCode()` together, always — never just one.
- Two equal objects (per `.equals()`) must return the same `hashCode()`; the reverse isn't required (hash collisions between unequal objects are fine).
 - `toString()` should return a human-readable summary for debugging/logging — the default `Object.toString()` prints an unhelpful class name + hash code.
- Java 16+ records auto-generate correct `equals`/`hashCode`/`toString` from the record's components.

Common Pitfalls

- Overriding `equals()` without overriding `hashCode()`, silently breaking `HashMap`/`HashSet` lookups for that type.
- Forgetting `@Override` and accidentally overloading instead of overriding due to a mismatched parameter type or a typo in the method name.
- Assuming static methods are polymorphic — they are resolved by the reference's declared type at compile time, not the runtime object type ("method hiding," not overriding).
- Making every field public "to keep it simple," defeating encapsulation and removing the ability to validate or change internal representation later.
- Calling `super(...)` after other statements in a subclass constructor — it must be the very first statement, or the compiler rejects it.
- Choosing an abstract class when an interface would allow implementing multiple contracts, unnecessarily locking a class into a single-inheritance chain.
- Relying on the default `toString()` output in production logs, getting an uninformative `ClassName@hexHash` string instead of meaningful data.

Key Takeaways

- Encapsulation: keep fields private, expose behavior through methods so internal representation can change safely.
- Inheritance is single for classes (extends one class) but multiple for interfaces (implements many).
- Overloading is resolved at compile time by parameter list; overriding is resolved at runtime by the object's actual class.
- Abstract classes share implementation across a family of related types; interfaces define a capability contract across unrelated types.
- Static members belong to the class and cannot access instance state directly — they are not polymorphic.
- Always override `equals()` and `hashCode()` together, or neither — breaking the contract corrupts hash-based collections silently.

Learn More

Oracle Java Tutorials — Classes and Objects

<https://docs.oracle.com/javase/tutorial/java/javaOO/classes.html>

Oracle Java Tutorials — Interfaces and Inheritance

<https://docs.oracle.com/javase/tutorial/java/landl/index.html>

GeeksforGeeks — equals() and hashCode() in Java

<https://www.geeksforgeeks.org/java/equals-hashcode-methods-java/>